

New Processes for New Horizons: The Incremental Commitment Model

Barry Boehm

University of Southern California
941 W. 37th Street SAL 326
Los Angeles, CA 90089
1-213-740-8163

boehm at usc.edu

Jo Ann Lane

University of Southern California
941 W. 37th Street SAL 318
Los Angeles, CA 90089
1-858-945-0099

jolane at usc.edu

ABSTRACT

The wide variety of software-intensive systems needed to support the new horizons of evolving technology, system and software complexity, high dependability, global interoperability, emergent requirements, and adaptability to rapid change make traditional and current one-size-fits-all process models infeasible. This tutorial presents the process framework, principles, practices, and case studies for a new model developed and being used to address these challenges. It has a series of risk-driven decision points that enable projects to converge on whatever combination of agile, plan-driven, formal, legacy-oriented, reuse-oriented, or adaptive processes that best fit a project's situation. The tutorial discusses the decision table for common special cases; exit ramps for terminating non-viable projects; support of concurrent engineering of requirements, solutions and plans; and evidence-based commitment milestones for synchronizing the concurrent engineering. The tutorial will include case studies and exercises for participants' practice and discussion.

Categories and Subject Descriptors

D.2.9 [Software Engineering]: Management – *life cycle*

H.1.1 [Information Systems]: Models and Principles – *value of information*

K.6.1 [Computing Milieux]: Management of Computing and Information Systems – *project and people management (systems analysis and design)*

General Terms

Management, Economics, Experimentation, Verification.

Keywords

Incremental Commitment Model, hardware-software-human factors integration, feasibility evidence, risk management.

1. TUTORIAL SUMMARY

One of the top recommendations to emerge from a major U.S. government Software Strategy Summit was to find ways of better integrating software engineering into the systems engineering and acquisition process. Concurrently, a U.S. National Research Council (NRC) study was addressing the problem of better

integrating human factors into the systems engineering and acquisition process. This tutorial presents a model that emerged from these and related efforts that shows promise of improving these integrations. This model, called the Incremental Commitment Model (ICM), organizes systems engineering, software engineering, and acquisition processes in ways that better accommodate the different strengths and difficulties of hardware, software, and human factors engineering approaches. It also provides points at which they can synchronize and stabilize, and at which their risks of going forward can be better assessed and fitted into a risk-driven stakeholder resource commitment process.

This tutorial focuses on a process framework that addresses the development and evolution of small relatively independent systems to large, complex systems that are often part of one or more system of systems (SoS). The tutorial presents the background on the development of the ICM, what the ICM is, and how to apply the ICM from the small single system to the global SoS perspective. It begins by summarizing trends that have caused difficulties for current systems engineering, software engineering, and acquisition processes (complex multi-owner systems of systems (SoS), emergent requirements, rapid change, reused components, high assurance of qualities) and principles underlying the ICM that better address these trends (stakeholder satisficing, incremental and evolutionary growth of system definition and stakeholder commitment, iterative development, concurrent definition and development, and risk management). It then presents several complementary views of the ICM, discusses their implications with respect to acquisition and engineering practices and personnel career paths, and assesses project performance with respect to use of the principles. The principles can also be used to avoid the negative effects of common misinterpretations of other current models such as the V, spiral, and Rational Unified Process (RUP) models.

If hardware-software-human factors integration problems are difficult today, they will present formidable problems for the future if not adequately addressed. Some trends that will exacerbate such integration problems are complex multi-owner SoSs that are part of global enterprises, emergent requirements, rapid changes, reused components, and high assurance of qualities. We have performed several analyses to determine the kind of process that would satisfactorily address these challenges. As part of a U.S. NRC study, we analyzed the strengths and difficulties of current process models. Each had strengths, but needed further refinements to address all of the five challenges above. The most important conclusion, though, was that there

were key process principles that address the challenges, and that forms of the models were less important than their ability to adopt the principles. These key principles are: 1) commitment and accountability of success-critical stakeholders, 2) stakeholder satisficing based on success-based negotiations and tradeoffs, 3) incremental and evolutionary growth of system definition and stakeholder commitment, 4) iterative system development and definition, 5) concurrent system definition and development allowing early fielding of core capabilities, continual adaptation to change, and timely growth of complex systems without waiting for every requirement and subsystem to be defined, and 6) risk management – risk driven anchor point milestones which are key to synchronizing and stabilizing all of this concurrent activity.

The ICM builds on the strengths of current process models: early verification and validation concepts in the V-model, concurrency concepts in the Concurrent Engineering model, lighter-weight concepts in the Agile and Lean models, risk-driven concepts in the spiral model, the phases and anchor points in the Rational Unified Process (RUP), and recent extensions of the spiral model to address SoS acquisition.

In comparison to the software-intensive RUP, the ICM also addresses hardware and human factors integration. It extends the RUP phases to cover the full system life cycle. And in comparison to the sequential waterfall and V-model, the ICM explicitly emphasizes concurrent engineering of requirements and solutions; establishes Feasibility Rationales as pass/fail milestone criteria; enables risk-driven avoidance of unnecessary documents, phases, and reviews; and provides support for a stabilized current-increment development concurrently with a separate change processing and rebaselining activity to prepare for appropriate and stabilized development of the next increment.

It also shows how projects can be organized into stabilized build-to-specification increments that fit current legacy contracting methods, along with concurrent agile change-adaptation and verification and validation (V&V) functions that need to use alternative contracting methods. Addressing changes of this nature will be important, particularly if organizations are to realize the large potential value offered by investments in future network-centric SoSs and global enterprises.

As part of the analysis done to support the development of the ICM, several excellent case studies that applied versions of the ICM process were identified. These case studies include a remotely piloted vehicle, a port security system, and a commercial medical infusion pump development and are highlighted in the tutorial.

Another set of successful projects that have applied the critical success factor principles is the annual series of Top-5 software-intensive systems projects published in CrossTalk. The “Top-5 Quality Software Projects” were chosen annually by panels of leading experts as role models of best practices and successful outcomes. The tutorial shows how the ICM principles were applied in these successful software projects.

2. REFERENCES

The tutorial builds upon the following references.

[1] Boehm, B. (July 1996). Anchoring the Software Process. Software. pp. 73-82.

[2] Boehm, B. (2000) “Unifying Software Engineering and Systems Engineering”, Computer, March 2000, pp. 114-116.

[3] Boehm, B., (2008) Making a Difference in the Software Century, IEEE Computer, March 2008, pp. 32-38.

[4] Boehm, B., and Bhuta, J. (2008) Balancing Opportunities and Risks in Component-Based Software Development, IEEE Software, November/December 2008, pp. 56-63.

[5] Boehm, B., Egyed, A., Kwan, J., Port, D., Shah, A., and Madachy, R. (July 1998). Using the WinWin Spiral Model: A Case Study. Computer. pp.33-44.

[6] Boehm, B., and Lane, J. (May 2006). 21st Century Processes for Acquiring 21st Century Systems of Systems. CrossTalk. pp. 4-9.

[7] Boehm, B., and Lane, J. (2007) Using the Incremental Commitment Model to Integrate System acquisition, Systems Engineering, and Software Engineering, CrossTalk, October 2007, pp. 4-9.

[8] Boehm, B., Valerdi, R., and Honour, E. (2008) The ROI of Systems Engineering: Some Quantitative Results for Software-Intensive Systems, Systems Engineering, Vol. 11, No.3, pp. 221-234.

[9] Clements, P. et al. (2002). Evaluating Software Architectures, Addison Wesley.

[10] CrossTalk (2002-2005), “Top Five Quality Software Projects,” Jan 2002, July 2003, July 2004, Sept 2005.

[11] Dahmann, J., Lane, J., Rebovich, G., and Baldwin, K. (2008); A Model of Systems Engineering in a System of Systems Context, Proceedings of the Sixth Conference on Systems Engineering Research.

[12] Hopkins, R., and Jenkins, K. (2008) Eating the IT Elephant: Moving from Greenfield Development to Brownfield, IBM Press.

[13] Kruchten, P. (1999). The Rational Unified Process, Addison Wesley.

[14] Lane, J and Boehm, B. (2008); System of Systems Lead System Integrators: Where do They Spend Their Time and What Makes Them More or Less Efficient, Systems Engineering Journal, Vol. 11, No. 1.

[15] Lane, L. and Boehm, B. (2007); Modern Tools to Support DoD Software Intensive System of Systems Cost Estimation, Data and Analysis Center for Software, a Department of Defense Information Clearinghouse.

[16] Maranzano, J. et al. (2005) Architecture reviews: Practice and experience. IEEE Software, March/April 2005.

[17] Pew, R. W., and Mavor, A. S., (2007). Human-System Integration in the System Development Process: A New Look, National Academy Press.

[18] Reichtin, E. (1991). Systems Architecting, Prentice Hall.

[19] Royce, W. E. (1998) Software Project Management. Addison Wesley.

[20] Royce, W. W. (1970). Managing the development of large software systems: concepts and techniques, Proceedings, Wescon 1970.